

Optical flow you can query: calibrated confidence and 27x cheaper repeat access for 2.6% of mean accuracy

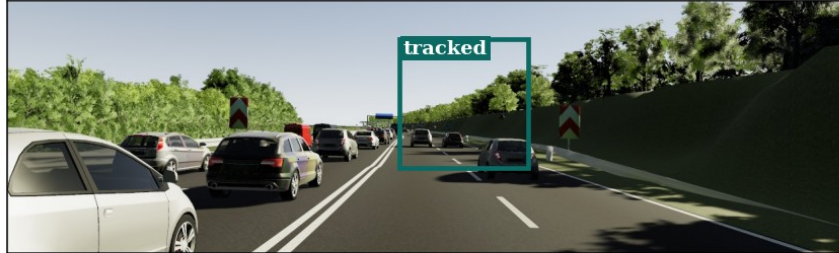
Shriman Raghav Srinivasan

MS Robotics, Northeastern University | Field Robotics Lab

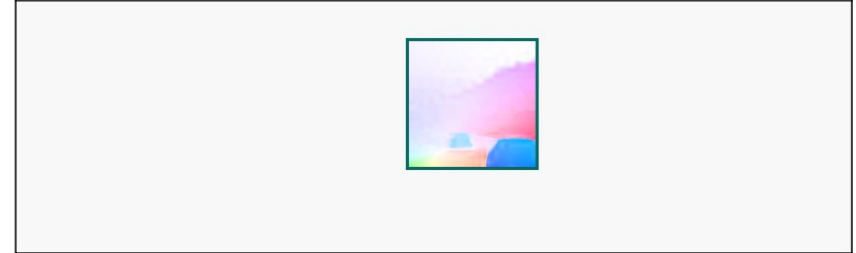
Final project presentation | 12 August 2026

A fast platform cannot afford the frame it is given

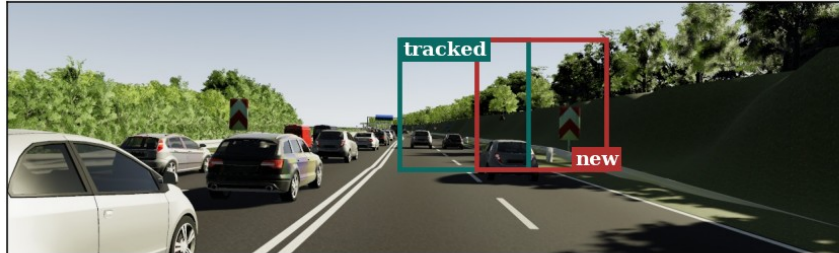
S1 First encounter — something worth flowing enters the field of view



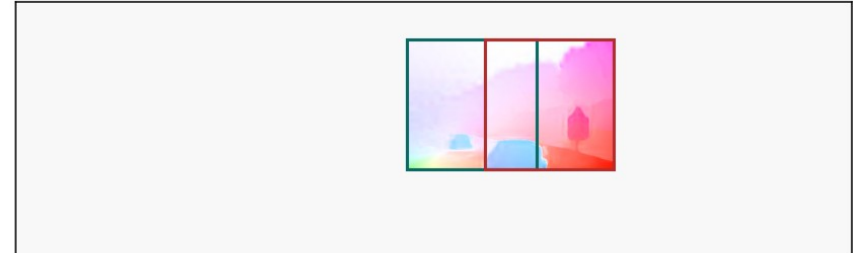
flow returned — computed only inside the regions



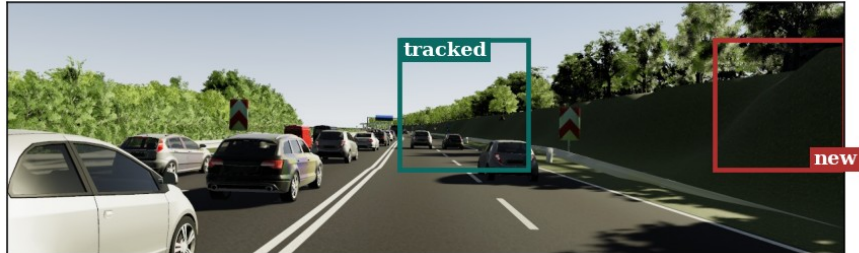
S2 Turn — the tracked region now overlaps a second object



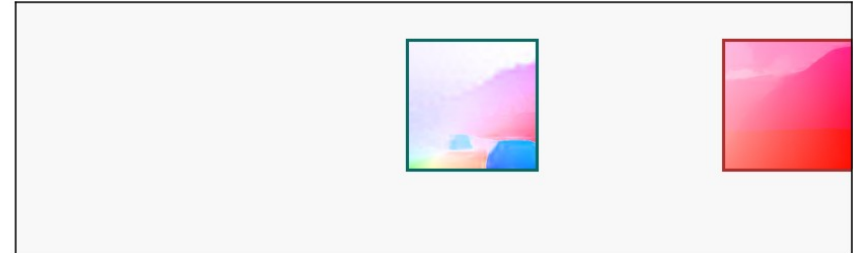
flow returned — computed only inside the regions



S3 New object — a second region appears in a frame already being processed



flow returned — computed only inside the regions



A dense output forecloses the two questions that platform actually asks

Where exactly? A corner tracked to (312.7, 188.2) cannot be requested. The pixel grid is a sampling artefact; the motion underneath it is continuous.

And again, on a frame already processed? A second question about the same pair costs the entire computation a second time, because nothing is kept between calls.

Meanwhile every consumer discards most of the answer. Registration, sparse tracking and survey mosaicking each select a few hundred points and throw away the other 478,000.

On constrained hardware, computing 479,232 correspondences in order to consume 800 is the difference between operating in real time and not.

Can the output stage be made queryable without surrendering accuracy or speed?

Can NeuFlow v2's fixed upsampler be replaced by a decoder evaluated at arbitrary continuous coordinates, and what does that cost?

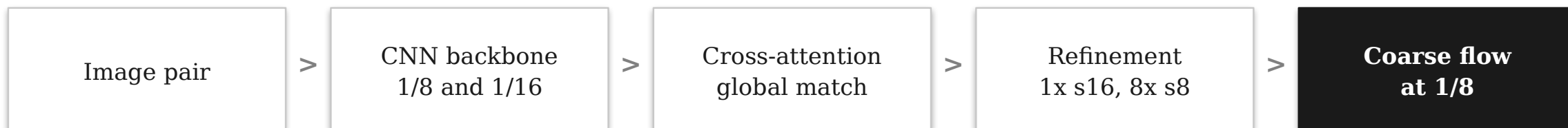
Approach. Freeze everything in v2 that understands motion. Change only the last stage.

Consequence. Cost scales with the number of queries, $O(N)$, not with image area, $O(H \times W)$.

Test. One variable per training run, and a front end verified unchanged.

v3 keeps NeuFlow v2 frozen and replaces only its fixed upsampler

Phase 1 once per frame pair, 33 ms, cached



Phase 2 once per query batch, 1.25 ms, $O(N)$



The output is a convex combination of neighbouring coarse-flow values, so the decoder cannot predict motion its inputs do not locally support.

The front end is verified bit-identical to v2, so the comparison isolates the decoder

Frozen front end. All 137 shared tensors verified bit-identical, per checkpoint.

Disjoint splits. Both evaluation scenes are excluded from every training set, checked at pair and frame level before each run.

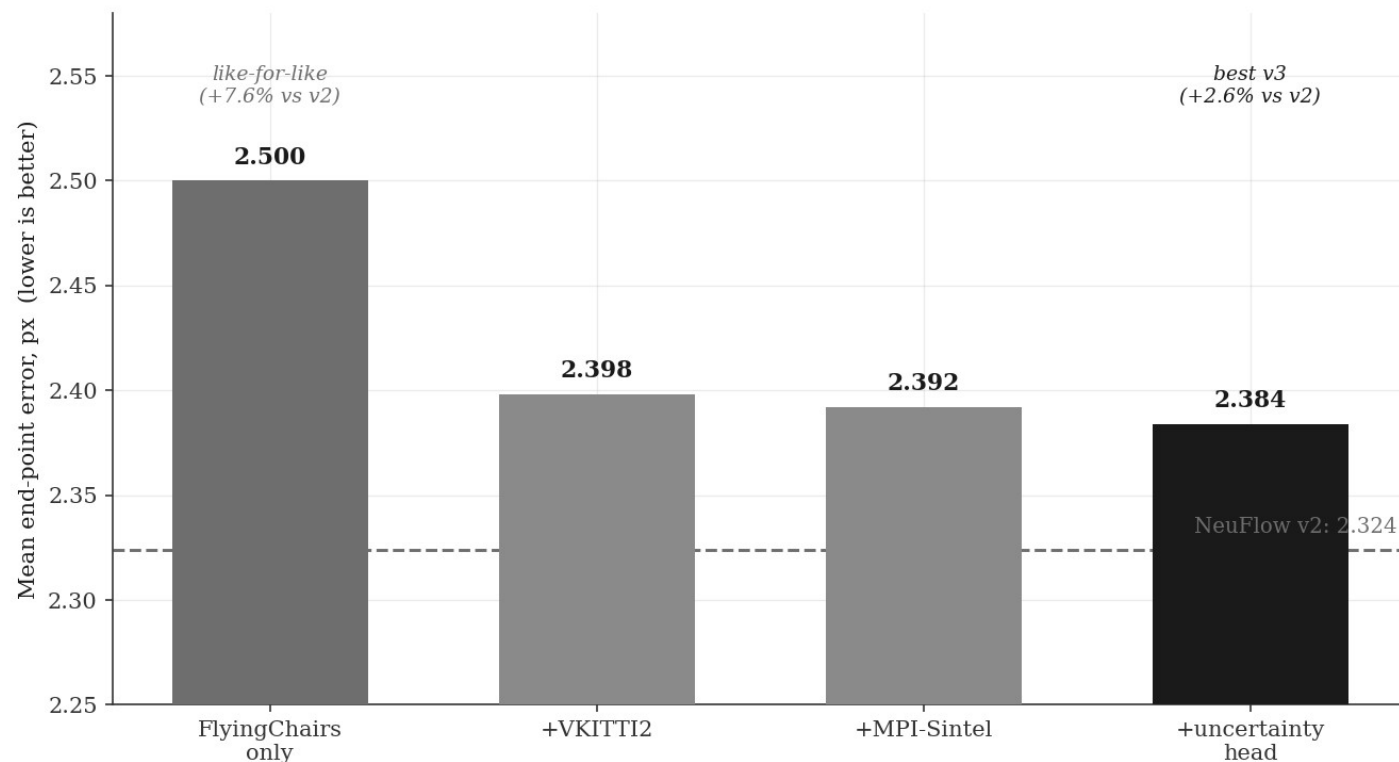
One variable per run. Same seed, schedule, batch size and query budget throughout.

Evaluation

Held-out pairs	1,174
Valid pixels	460.6 M
Cross-domain	TartanAir
Hardware	RTX 4060
Precision	fp16

Zero-initialised, the decoder is exactly bilinear upsampling: training can only move away from a known-good operating point by learning something.

The decoder costs 2.6 percent of mean error, from a model 13 percent smaller



THE COST, STATED FIRST

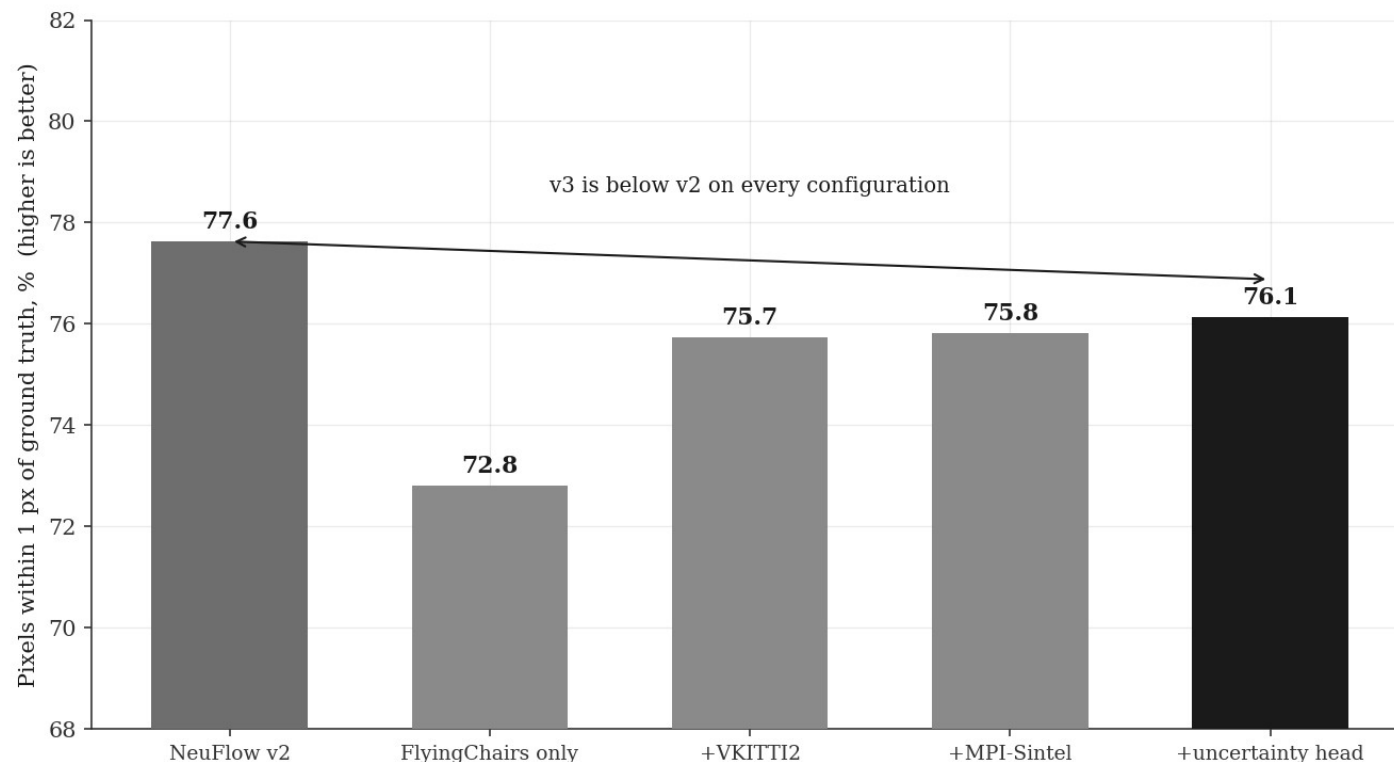
Every v3 configuration sits above v2.

Like-for-like on FlyingChairs alone: 2.500, or 7.6% behind.

7.83 M parameters against 9.03 M.

2.384
against 2.324

The cost concentrates in sub-pixel precision, not in large errors



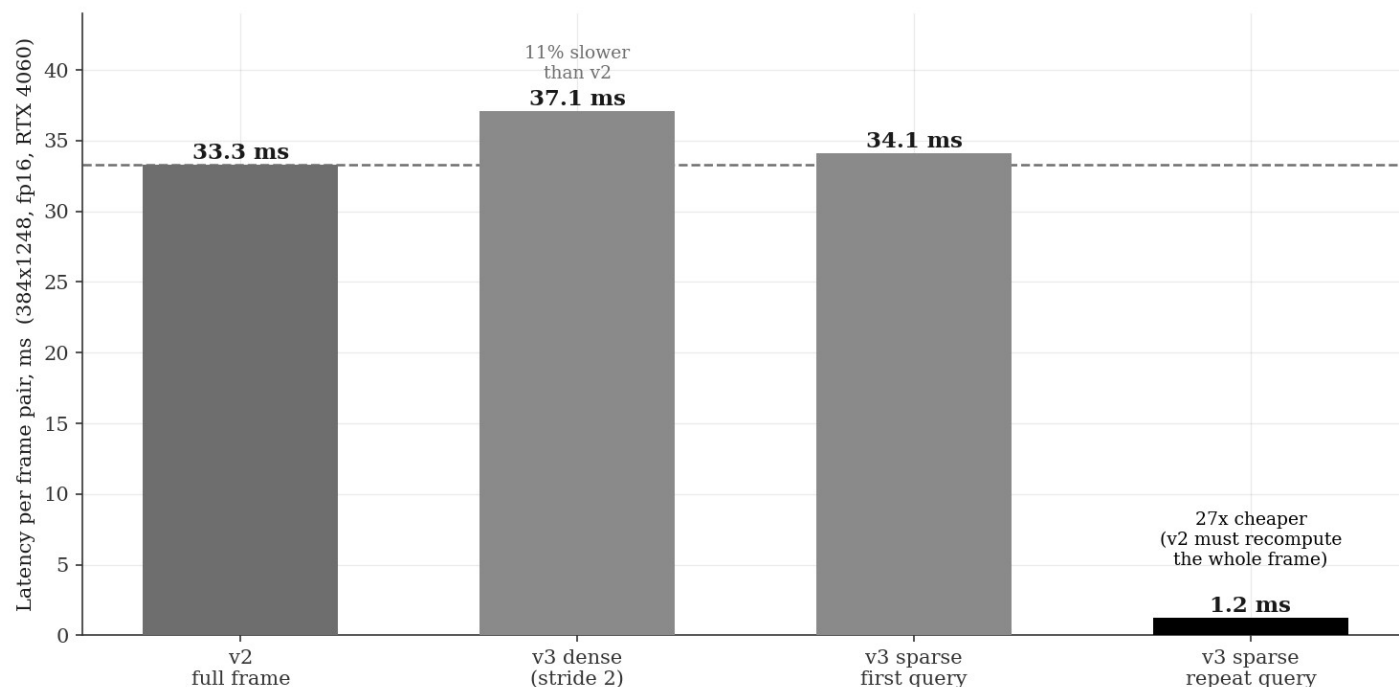
DIAGNOSED CAUSE

The decoder's finest input is 1/8 scale, so evidence barely varies inside an 8x8 cell. The upsampler it replaces reads the full-resolution frame.

A Fourier positional encoding changed nothing, which rules out a missing position signal.

**1.5 points below v2
on 1 px accuracy**

A repeat query on a cached frame costs 1.25 ms, against 33.3 ms for a fresh pass



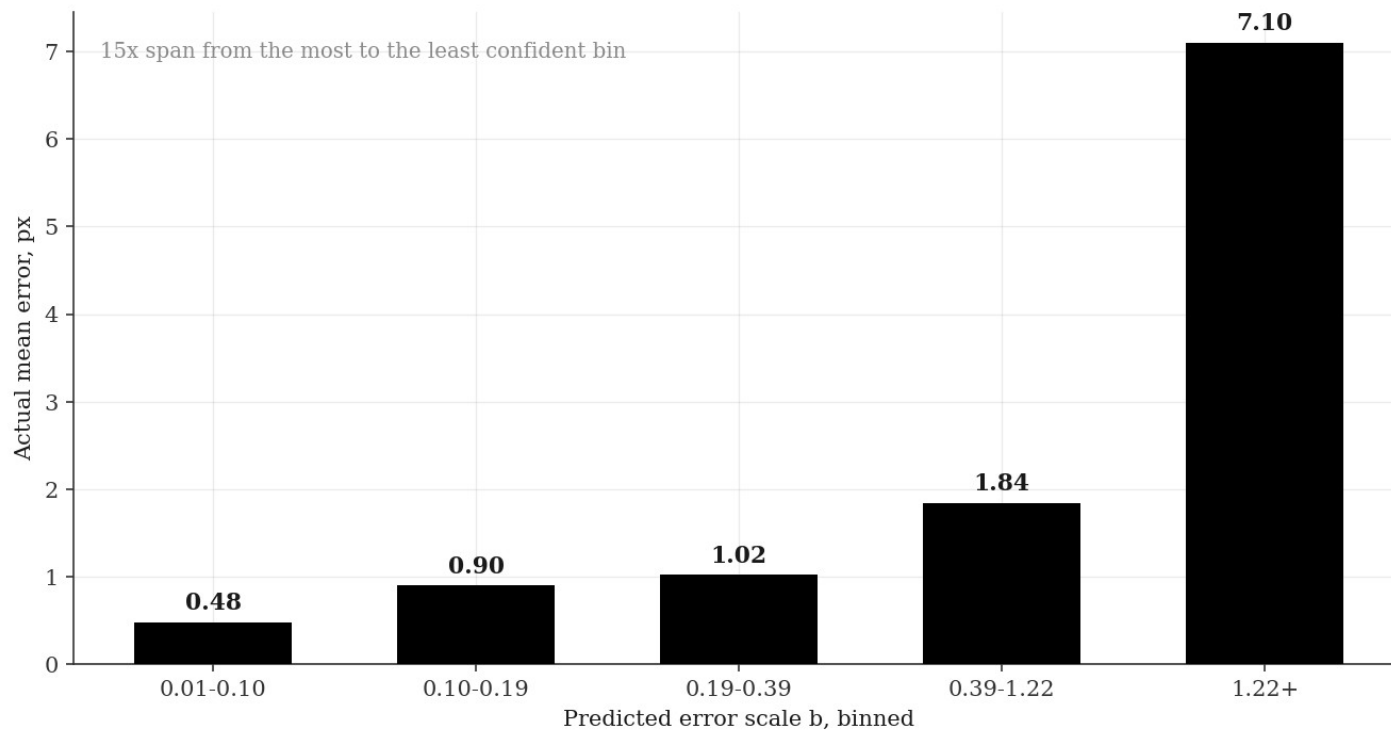
WHERE THE WIN IS

A first sparse query is level with v2: both pay the coarse pass, and global matching needs the whole image, so it cannot be skipped.

v2 keeps no state, so a second question costs a full recomputation.

27x on repeat access

Predicted confidence tracks real error monotonically across a 15-fold span



WHY IT IS USABLE

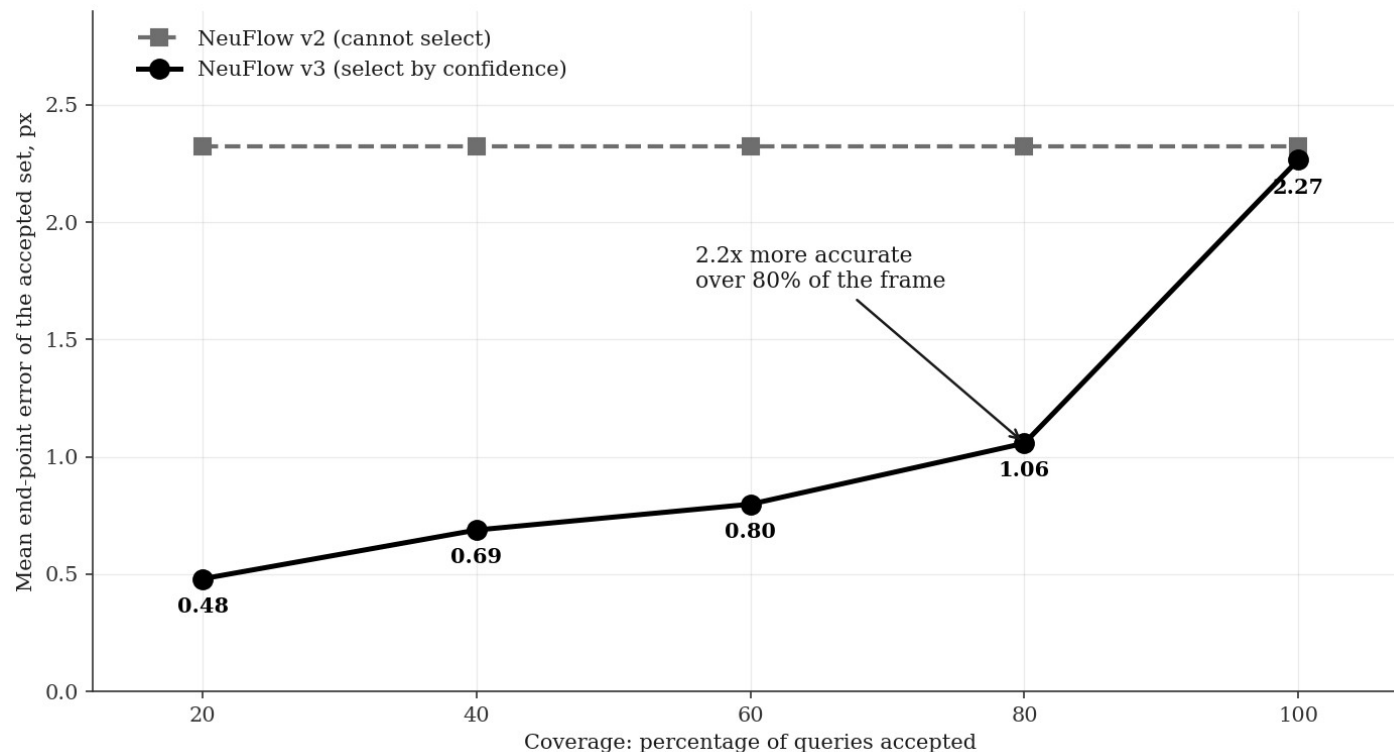
Weight correspondences in RANSAC by predicted scale.

Reject unreliable matches before they reach a solver.

v2 emits flow alone. There is no equivalent quantity to compare against.

**0.48 px to 7.10 px
across five bins**

Abstaining on 20 percent of queries makes v3 2.2x more accurate than v2



WHY THIS IS THE RIGHT COMPARISON

Mean error over all pixels is one operating point, and the only one a model without confidence has.

Registration picks a few hundred points anyway, so declining to answer costs nothing it would have used.

**1.06 px at 80% coverage
against v2 at 2.32**

Processing 14 percent of the frame costs 0.034 px and runs 4.4x faster

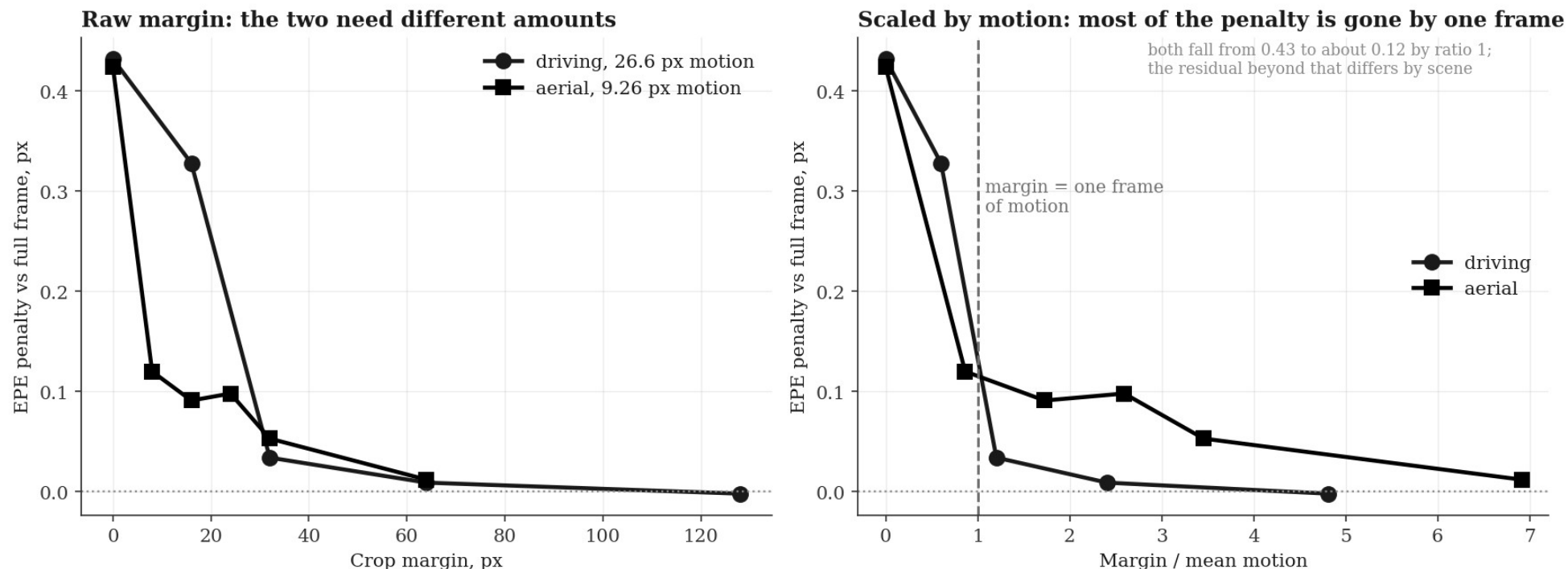
Region processed	Area	Latency	Speedup	EPE in region
Full frame	100%	33.3 ms	1.0x	0.657
Region, no margin	7.9%	7.8 ms	4.3x	1.089
Region + 32 px margin	13.8%	7.6 ms	4.4x	0.691
Region + 64 px margin	20.1%	8.1 ms	4.1x	0.667
Region + 128 px margin	34.2%	9.9 ms	3.4x	0.655

Design rule

margin $\approx$ expected motion
 $\approx$ speed / frame rate

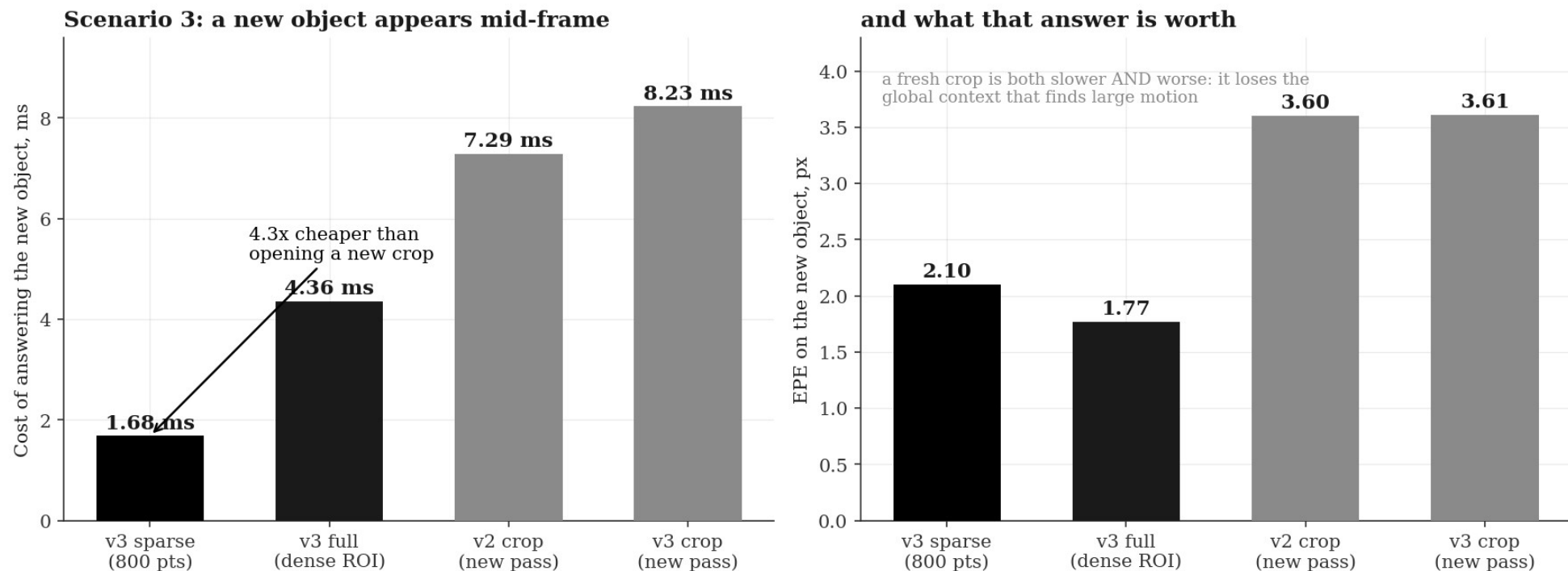
The margin is not optional. Without one, error rises 65 percent and a quarter of large-motion pixels fail outright, because global matching loses the context it uses to locate distant correspondences.

The margin a region needs is set by motion, not by image size



Driving averages 26.6 px of motion and needs about 32 px of margin. Aerial averages 9.26 px and needs about 8. Both shed most of the penalty by one frame of motion.

A region discovered mid-frame costs 1.7 ms, against 7.3 ms for a fresh crop



The cached coarse state answers a question the platform did not have when the frame arrived. A cropped pipeline must run a whole new pass, and is less accurate doing it.

On a domain neither model trained for, v3 leads at all seven margin settings

Crop margin	NeuFlow v2	NeuFlow v3
full frame	0.781	0.777
0 px	1.205	1.176
8 px	0.901	0.881
16 px	0.872	0.865
24 px	0.879	0.854
32 px	0.834	0.814
64 px	0.793	0.778

HOW MUCH IT COUNTS

The only unconfounded comparison in the deck. Every other table is on a domain v3 trained on and v2 did not.

Consistent rather than large: 0.004 to 0.029 px, over 40 pairs and one seed.

v3 is still worse on large-motion failures, 7.2% against 6.5%.

The decoder earns its place when the consumer chooses its own points

USE THE DECODER WHEN

The consumer selects its own query points.

A frame is revisited after it has been processed.

Each correspondence needs a confidence value.

A position between pixels is required.

USE THE BASELINE WHEN

One complete dense field per frame is consumed, and nothing else is needed.

v2 is more accurate and 10% faster in exactly that mode, and remains the correct choice for it.

Selective prediction reframes the comparison: mean error over all pixels is the metric the decoder loses on, and the only one a model without confidence can report.

The accuracy gap has a single cause, and three experiments converge on it

Accuracy. The decoder's finest input is 1/8 resolution; the upsampler it replaces reads the full-resolution frame.

Sub-pixel querying. For the same reason, evaluation between pixels is presently closer to interpolation than inference. The interface is exact; the extra information recoverable is small.

Embedded validation. All latency is from a laptop GPU. The edge premise is a design target, not a result.

Statistical resolution. One seed per configuration. Differences below about 0.05 px are not resolved by this study.

Three independent nulls point at the same mechanism: Fourier encoding, sub-pixel querying, and reduced-resolution querying. The remedy is a full-resolution stem.

Conclusions

- 1. Queryable output is affordable.** Arbitrary continuous coordinates in $O(N)$, for 2.6% of mean accuracy, from a model 13% smaller.
- 2. Confidence is worth more than the mean.** Abstaining on 20% of queries gives 2.2x lower error over the remaining 80% of the frame.
- 3. Region margins are predictable in advance.** Margin $\approx$ speed / frame rate, verified across a threefold difference in motion scale.

Next: a full-resolution feature stem, then embedded measurement.

srinivasan.shrim@northeastern.edu | Code and report: github.com/shrirag10/Neuflowv3

References

Zhang, Z., Gupta, A., Jiang, H., & Singh, H. (2024). NeuFlow v2: High-Efficiency Optical Flow Estimation on Edge Devices. arXiv:2408.10161.

Jung, H., Hui, Z., Luo, L., Yang, H., Liu, F., Yoo, S., Ranjan, R., & Demandolx, D. (2023). AnyFlow: Arbitrary Scale Optical Flow with Implicit Neural Representation. CVPR.

Teed, Z., & Deng, J. (2020). RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. ECCV.

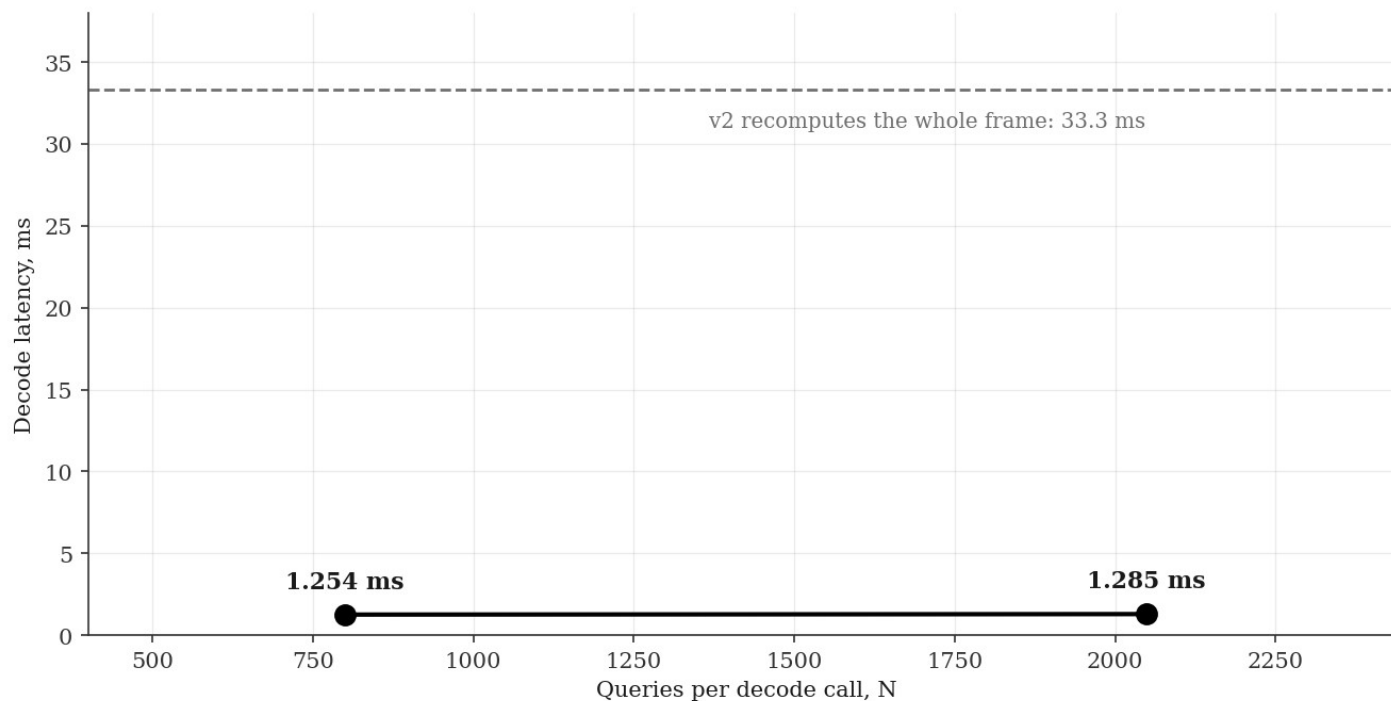
Yu et al. (2025). InfiniDepth: Implicit Neural Upsampling for Monocular Depth Estimation.

Kendall, A., & Gal, Y. (2017). What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? NeurIPS.

Cabon, Y., Murray, N., & Humenberger, M. (2020). Virtual KITTI 2. arXiv:2001.10773.

Wang, W., Zhu, D., Wang, X., Hu, Y., Qiu, Y., Wang, C., Hu, Y., Kapoor, A., & Scherer, S. (2020). TartanAir: A Dataset to Push the Limits of Visual SLAM. IROS.

Decode cost is flat to 2,048 queries: 2,048 points cost what 800 do

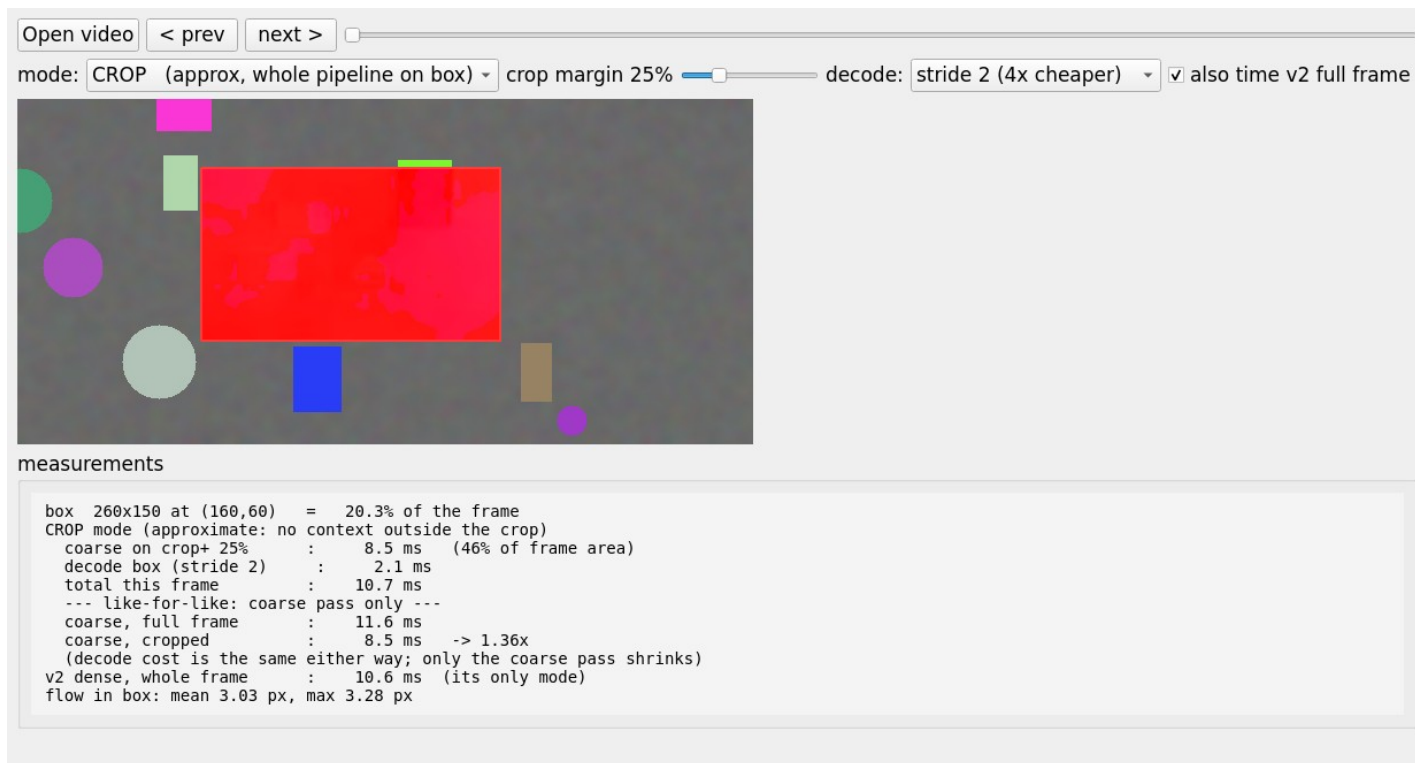


THE ENTIRE API

```
state = model.infer_coarse_state(  
    img0, img1)  
  
flow = model.decode_queries(  
    state, query_coords=q)  
  
flow, b = ... return_uncertainty=True
```

Launch-overhead bound below roughly 10,000 points, compute-bound above, where dense decoding is the better choice.

The decoder runs as an interactive tool: draw a region, get flow there



TWO TIMED MODES

QUERY decodes only inside the box against a full-frame coarse pass. Exact, and identical to the full-frame result inside the box.

CROP feeds only the box through the pipeline, so cost scales with area, at the price of the lost context.

Two of three original objectives were not met; the project delivers three others

Objective	Verdict	Evidence
Better accuracy than v2	not met	best 2.384 against 2.324; like-for-like 7.6% behind dense 10% slower; repeat query 27x cheaper every figure is from a laptop RTX 4060
Less compute than v2	partly	
Runs on edge devices	unproven	

WHAT IT DOES DELIVER

Queryable output

any continuous coordinate;
sparse equals dense exactly

Cheap repeat access

1.25 ms per further query
batch on a cached frame

Calibrated confidence

a per-query error estimate,
monotonic against real error

Four questions I expect, answered

Why is flow defined between pixels at all?

Motion is continuous and the pixel grid is a sampling artefact. A corner tracked to (312.7, 188.2) is a real answer, and registration consumes it directly.

Why freeze the backbone?

To make the decoder the only variable. Any difference between v2 and v3 is then attributable to the output stage and nothing else.

Why is dense v3 slower than v2?

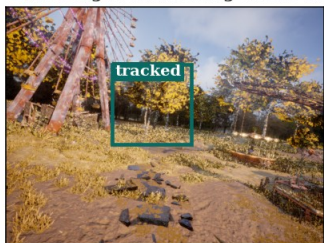
v2 upsamples with one convolution that shares work across neighbouring pixels. v3 answers each query independently, which gives up that sharing.

Is 2.6 percent worth the added complexity?

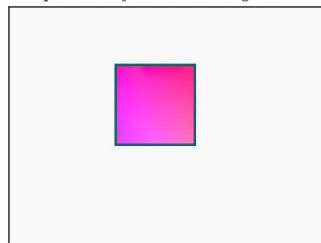
Not on its own. It is worth it if you need queryability, cheap repeat access, or a confidence value. For plain dense flow, v2 remains the better choice.

Same region geometry, same decoder, only the domain changes

S1 First encounter
something worth flowing enters the field of view



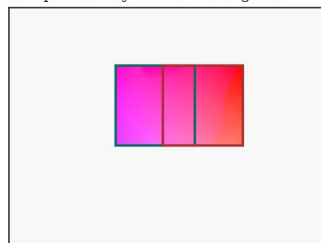
flow returned
computed only inside the regions



S2 Turn
the tracked region now overlaps a second object



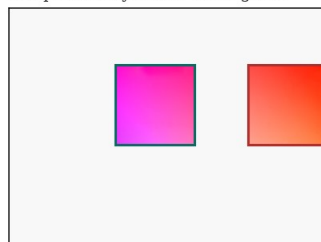
flow returned
computed only inside the regions



S3 New object
a second region appears in a frame already being processed



flow returned
computed only inside the regions



What is held fixed. Region geometry, margin, decode stride and checkpoint are identical to the driving figure on slide 2.

What changes. Mean inter-frame motion falls from 26.6 px to 9.26 px, about a third.

What to notice. Flow inside each region is much closer to uniform than in the driving frames, because a drone rotating through a static scene produces near-constant displacement across a small window.

Why it matters. No model in this study trained on this imagery, which is what makes the head-to-head on slide 15 worth having.